

Assignment 6 Branch Target Buffer

The sixth assignment extends the pipeline with a Branch Target Buffer (BTB). Your task is to implement a BTB, as presented in the ADS I lecture (slide 6-48), in combination with a dynamic 2-bit branch predictor (slide 6-47). A BTB is a hardware component used in modern processors to improve the efficiency of branch instructions. The BTB acts as a lookup table for target addresses. When the control flow encounters a branch instruction, the BTB checks if it contains an entry for this branch and what outcome the prediction scheme used by the BTB predicts. This allows the processor to quickly fetch the next instruction without waiting for a full evaluation of the branch. If the prediction is correct, the processor continues execution without delay. If it is wrong, the processor rolls back and resumes execution at the correct target PC. In both cases, the prediction is updated to reflect the current behavior of the program. This way, a BTB helps to reduce pipeline stalls and improves overall performance. The BTB serves as a cache for target addresses. Therefore, we can utilize strategies known from data caches, such as associativity (cf. slide 7-23), to make the BTB more efficient. The BTB should only be applied to conditional branch instructions (beq, bne, blt, bge, bltu, bgeu). Unconditional jumps (jal, jalr) should always be handled as taken branches without consulting the BTB.

Structure of the Project

This task uses your implementation from task 5 (5-stage pipeline supporting branch and jump instructions) as a starting point. Make sure that your core is working correctly before starting with this assignment.

Task 6.1 Preparation

Before you start with the implementation, please answer the following questions:

1. How does a higher associativity affect the performance of a cache?
2. What's the best initial state for the 2-bit predictor FSM? What effects would different initial states have in regular program patterns (e.g., loops)?

It might also help you to think about the following details before starting with your implementation.

1. Find a way to implement a structure of registers with 8 sets and 2 ways. Standard memory classes might not be a good option here.
2. How can the number of index bits be determined?
3. How many bits does the tag of each BTB entry need in a configuration with 8 sets and 2 ways?

Task 6.2 Specification

Enhance your RISC-V core with a 2-way set-associative Branch Target Buffer (BTB) with 8 sets. Your design should operate as an independent module, interfacing with the rest of the processor through a predefined interface.

The BTB consists of 8 sets, each containing 2 ways, making it a 2-way set-associative structure. Each entry in the BTB should include:

- A valid bit to indicate whether the entry contains usable data.
- A tag to identify the branch instruction associated with the entry.

- A branch target address, which provides the predicted next program counter (PC) when the branch is taken.
- A 2-bit predictor state to indicate whether the branch is predicted to be taken or not.

Each entry in the BTB contains a 2-bit saturating counter (similar to the one shown on slide 6-47 in the ADS I lecture slides) to predict whether the branch is likely to be taken or not taken. In a 2-bit counter, a prediction must be wrong twice before it is changed. This enhancement allows the BTB not only to predict the target address of a branch but also to predict whether the branch will be taken or not. The branch prediction mechanism implements a 2-bit finite state machine (FSM) with four states:

- strongTaken
- weakTaken
- weakNotTaken
- strongNotTaken

The BTB uses bits [4:2] of the input program counter to index into one of the 8 sets (bits [1:0] are always zero, due to RV32I instruction alignment). For each set, the two ways should be checked for a tag match. If a matching tag is found and the corresponding entry is valid, the BTB should output the predicted branch target address. If no valid match is found, the BTB should indicate that no prediction is available (valid := false.B). In this case, a new entry will be written after the branch target has been calculated in EX stage (update signals). Do not forget to also initialize the 2-bit predictor FSM for a new entry. Since each set has two ways, the design must implement a least recently used (LRU) replacement policy. This policy ensures that when a new entry is added to a full set, the entry that has not been used for the longest time will be evicted.

The BTB module uses the following I/O signals:

Inputs:

- PC: A 32-bit program counter representing the address of the branch instruction being fetched or executed.
- update: A 1-bit signal indicating whether the BTB should be updated with new information.
- updatePC: A 32-bit program counter associated with the branch instruction being updated.
- updateTarget: A 32-bit branch target address to be stored in the BTB.
- mispredicted: A 1-bit signal indicating whether the prediction turned out to be incorrect during execution (used to update the predictor).

Outputs:

- valid: A 1-bit signal indicating whether the BTB has a valid prediction for the provided program counter.
- target: A 32-bit signal representing the predicted branch target address when a valid prediction exists.
- predictTaken: A 1-bit signal indicating whether the branch is predicted to be taken or not.

When the update signal is asserted, the BTB should update the appropriate set with the updatePC and updateTarget values. In addition to updating the target address and tag, the predictor's state must also be initialized. If both ways in the set are already in use, the LRU replacement policy should be used to determine which entry to evict. An update is sent to the BTB two clock cycles as soon as the actual branch target is calculated in the execute phase. Therefore, it is important to ensure that the update mechanism still selects the correct entry when a new target is written and the 2-bit predictor FSM is updated.

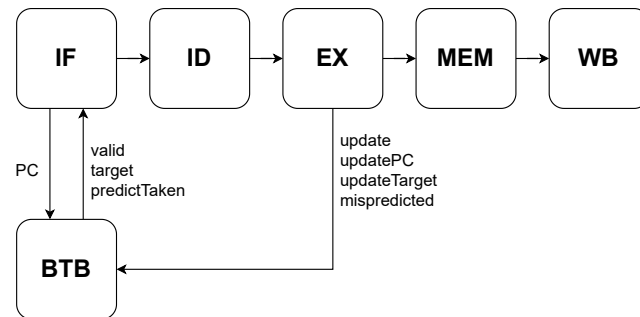


Figure 1: I/O Signals

Task 6.3 Implementation

Implement a 2-way set-associative BTB according to the specification above and integrate it into your core design. Structure your core in a modular way, so that you can easily switch between the static prediction scheme from the previous task and the BTB-based dynamic prediction scheme.

Task 6.4 Performance Evaluation

Evaluate the performance of your BTB implementation by measuring the branch prediction accuracy. Create a set of benchmark programs that include various branching patterns (e.g., loops, conditional branches) and run them on your processor core with the BTB enabled. Calculate the prediction accuracy as the ratio of correctly predicted branches to the total number of branches executed. Compare the performance of your processor core with and without the BTB to highlight the improvements achieved through branch prediction.

Task 6.5 Test your Implementation

Create test cases to verify your BTB's functionality. These tests should demonstrate the following:

- Correct predictions for program counters with valid entries.
- Handling updates to the BTB properly.
- Correct eviction behavior based on the LRU replacement policy.
- Demonstrate accurate state transitions of the FSM and correct predictions based on the current state.

A [RISC-V encoder](#) and a [RISC-V interpreter](#) might be helpful for you to achieve this task.